



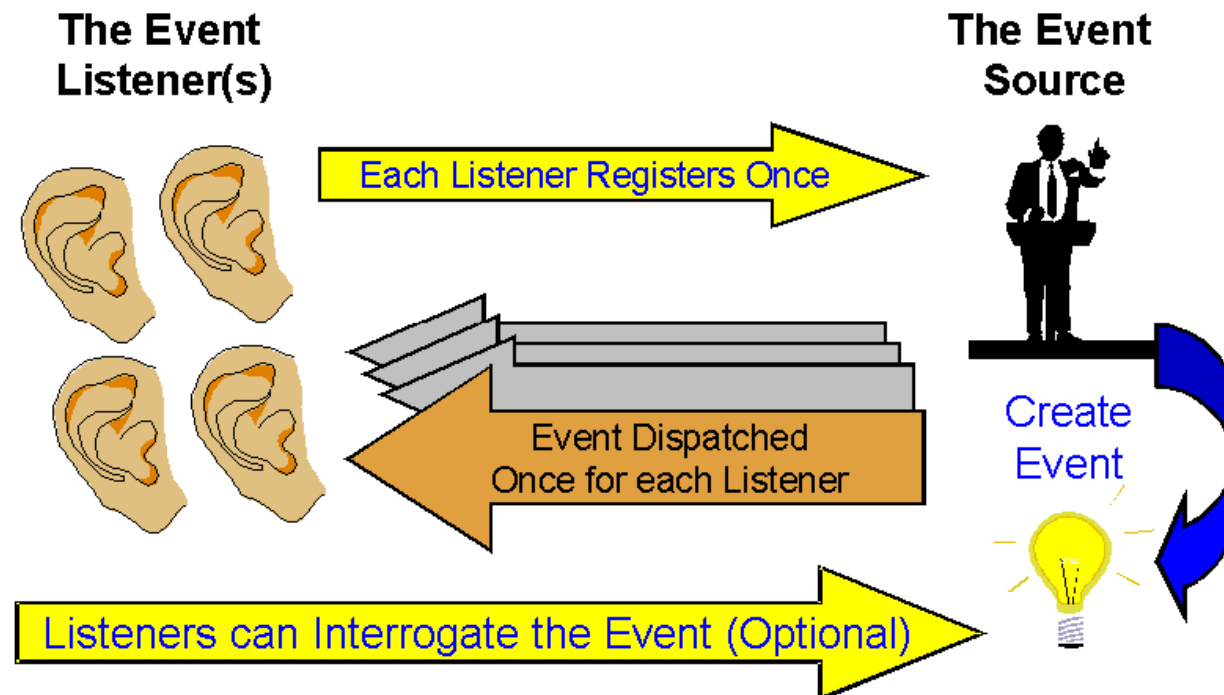
Eventos

Tokio.



O que são eventos?

Um evento é um acontecimento durante a execução de um programa. Muitas vezes são gerados a partir de interfaces gráficas (clique num botão, premir de uma tecla, seleção de uma opção numa lista...), mas também podem ocorrer noutras situações (recepção de uma mensagem de um servidor, temporizador que acaba, um erro despoletado...). Quando um evento é lançado, apenas os seus Listeners são alertados, evitando assim que outras estruturas o recebam e percam tempo a processá-lo. Um listener é uma classe que deverá ser registada no início do programa (ou quando necessário), função essa que começará a correr assim que o evento é lançado.





Como podemos usar eventos?

Para que uma classe possa receber um determinado evento, esta deve registrar-se como tal.

```
public void addTypeListener (TypeListener el )
```

Type será o nome do evento (Key, MouseMotion, ...) e el o Event Listener que deverá ser notificado do lançamento do evento. Todos os listeners que se registem com uma chamada semelhante a esta irão receber o despoletar do evento, a isto chama-se multicasting.

De seguida, deveremos ter os nossos Handlers implementados, isto é, as funções que vão processar o resultado do evento, reagindo de acordo com o que for pretendido. Os Handlers não podem ser extensos, correndo o perigo de a UI (em caso de eventos de UI) se tornar lenta e irresponsiva. Deverão fazer apenas o necessário, mesmo que deleguem outras tarefas para outras instâncias.

Outra chamada importante é o removeTypeListener, que nos permite remover o nosso interesse em receber aquele determinado evento quando não precisarmos dele ou formos destruir a presente instanciação da classe, por exemplo.

```
public void removeTypeListener(TypeListener el )
```



EventObject

As classes de eventos são muitas e não temos tempo para falar de todas, mas vamos mencionar as principais da hierarquia. No seu topo, está a EventObject, superclasse de todos os eventos. O seu construtor é:

EventObject(Object src)

src é o objeto que irá gerar o nosso evento. Esta classe tem 2 métodos: getSource() e toString(). getSource() retorna o objeto fonte do evento. toString() retorna o equivalente à string do evento.

Event Class	Description
ActionEvent	Generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
AdjustmentEvent	Generated when a scroll bar is manipulated.
ComponentEvent	Generated when a component is hidden, moved, resized, or becomes visible.
ContainerEvent	Generated when a component is added to or removed from a container.
FocusEvent	Generated when a component gains or loses keyboard focus.
InputEvent	Abstract superclass for all component input event classes.
ItemEvent	Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
KeyEvent	Generated when input is received from the keyboard.
MouseEvent	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
MouseEvent	Generated when the mouse wheel is moved.
TextEvent	Generated when the value of a text area or text field is changed.
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.



ActionEvent

Um ActionEvent é lançado quando um botão é premido, quando se faz um duplo clique num item de uma lista ou um item de um menu é selecionado. O ActionEvent define quatro constantes de inteiros que podem ser usadas para identificar qualquer modificador associado com o evento: ALT_MASK, CTRL_MASK, META_MASK e SHIFT_MASK. Existe outra constante, ACTION_PERFORMED, que identifica eventos de ação.

Esta classe tem 3 construtores:

`ActionEvent(Object src, int type, String cmd)`

`ActionEvent(Object src, int type, String cmd, int modifiers)`

`ActionEvent(Object src, int type, String cmd, long when, int modifiers)`

src é uma referência ao objeto que gera o evento. O tipo de evento é especificado por type, sendo a string de comando cmd. Os modificadores de argumentos indicam quais das teclas modificadoras foram premidas (ALT, CTRL, META e/ou SHIFT). O parâmetro when especifica quando o evento ocorreu. Podemos obter o nome do comando para invocar um ActionEvent através do método `getActionCommand()`, que devolve uma string. Quando um botão é premido, um ActionEvent é gerado com o nome do comando igual à label do botão para que possamos facilmente identificar. `getModifiers()` retorna um valor que indica quais modificadores foram premidos à criação do evento, enquanto `getWhen()` retorna o momento em que o evento ocorreu, através da sua timestamp.



AdjustmentEvent

Este evento é gerado por uma barra de scroll. Existem 5 tipos de eventos de ajustamento, na tabela abaixo. Existe ainda uma constante de int, `ADJUSTMENT_VALUE_CHANGED`, que indica que uma alteração ocorreu. O construtor destes eventos é o seguinte:

`AdjustmentEvent(Adjustable src, int id, int type, int val)`

`src` refere o objeto que gerou o evento, `id` especifica o tipo do evento. O tipo do ajustamento é especificado por `type`, enquanto o seu valor associado é `val`. `GetAdjustable()` retorna o objeto que gerou o evento. O tipo do adjustment pode ser obtido através de `getAdjustmentType()`, enquanto que o valor é obtido por `getValue()`.

BLOCK_DECREMENT	The user clicked inside the scroll bar to decrease its value.
BLOCK_INCREMENT	The user clicked inside the scroll bar to increase its value.
TRACK	The slider was dragged.
UNIT_DECREMENT	The button at the end of the scroll bar was clicked to decrease its value.
UNIT_INCREMENT	The button at the end of the scroll bar was clicked to increase its value.



ComponentEvent

Um ComponentEvent é gerado quando o tamanho, posição ou visibilidade de um componente é alterada. Existem 4 tipos destes eventos, na tabela. Estas são 4 constantes de inteiros.

COMPONENT_HIDDEN	The component was hidden.
COMPONENT_MOVED	The component was moved.
COMPONENT_RESIZED	The component was resized.
COMPONENT_SHOWN	The component became visible.

ComponentEvent é a superclasse, direta ou indiretamente, para ContainerEvent, FocusEvent, KeyEvent, MouseEvent e WindowEvent, entre outros.



ContainerEvent

Estes eventos são gerados quando um componente é adicionado ou removido de um container. Existem 2 tipos de container events, de acordo com a adição/remoção de elementos. Estes podem ser COMPONENT_ADDED e COMPONENT_REMOVED.

`ContainerEvent(Component src, int type, Component comp)`

src é uma referência para o container que gerou o evento, o seu tipo é mencionado por type e o componente adicionado ou removido é comp. Podemos obter uma referência para o container que despoletou o evento através de `getContainer()`. `getChildren()` retornar-nos-á uma referência ao componente adicionado/removido.



FocusEvent

Estes eventos são gerados quando um componente ganha ou perde o foco de input. Estes eventos são identificados pelos inteiros FOCUS_GAINED e FOCUS_LOST.

FocusEvent(Component *src*, int *type*)

FocusEvent(Component *src*, int *type*, boolean *temporaryFlag*)

FocusEvent(Component *src*, int *type*, boolean *temporaryFlag*, Component
other)

FocusEvent(Component *src*, int *type*, boolean *temporaryFlag*, Component
other,
FocusEvent.Cause *what*)

src indica a fonte do evento, o seu tipo é indicado por *type*. *temporaryFlag* refere se o evento é temporário ou não, sendo que os temporários são resultado de outra operação da interface do utilizador. Por exemplo, o utilizador está a inserir dados num campo de texto e move o rato para ajustar a scroll bar, é uma perda de foco temporária. Um evento de foco ganho envia no campo *other* o componente que tenha perdido o foco, bem como o evento de foco perdido irá enviar em *other* o componente que ganhou o foco.



InputEvent

Esta classe abstrata é subclasse de `ComponentEvent` e é superclasse para eventos de input em components. As suas subclasses são `KeyEvent` e `MouseEvent`.

A classe tem várias constantes que representam os modificadores que poderão estar associados com os eventos. `InputEvent` define os seguintes oito valores para representar modificadores:

ALT_DOWN_MASK	BUTTON2_DOWN_MASK	META_DOWN_MASK
ALT_GRAPH_DOWN_MASK	BUTTON3_DOWN_MASK	SHIFT_DOWN_MASK
BUTTON1_DOWN_MASK	CTRL_DOWN_MASK	

Caso queiramos verificar algum deles, usamos os métodos `isTypeDown`, em que `Type` é o nome do nosso modificador. Também é possível obter um valor que contém todos estes modificadores, através do método `getModifiersEx()`.



ItemEvent

Os ItemEvents são gerados por alterações em checkboxes ou itens de listas que são selecionados, bem como itens de menu selecionáveis que são selecionados. Existem 2 tipos de item events, tendo em conta as variáveis DESELECTED e SELECTED. Além disso, a classe também tem a constante ITEM_STATE_CHANGED que significa uma mudança de estado.

`ItemEvent(ItemSelectable src, int type, Object entry, int state)`

O item que gerou o evento é passado em entry. O estado atual desse mesmo item é comunicado em state.



KeyEvent

Estes eventos são gerados em input de teclado. Existem 3 tipos de eventos, KEY_PRESSED, KEY_RELEASED e KEY_TYPED. Os primeiros 2 eventos são gerados quando uma tecla é premida ou libertada, enquanto o último apenas acontece quando um caractere é gerado. Lembremo-nos que nem todas as teclas resultam em caracteres.

Existem muitas outras constantes definidas nesta classe. Desde VK_0 até VK_9, VK_A até VK_Z definem números e letras. As constantes VK definem códigos virtuais para as teclas e são independentes de qualquer modificador como o CTRL, SHIFT ou ALT.

`KeyEvent(Component src, int type, long when, int modifiers, int code, char ch)`

src é uma referência à fonte do evento, o tipo é passado em type. A timestamp do evento é passado em when, os modificadores estão indicados em modifiers. O caractere equivalente à tecla (ou combinação) passa em ch. Se não existir caractere válido, então ch vai ter CHAR_UNDEFINED. Em eventos KEY_TYPED, code tem VK_UNDEFINED.



MouseEvent

Os eventos lançados pelo rato estão na tabela ao lado. Dois dos métodos mais usados nesta classe são `getX()` e `getY()`. Também é possível usar `getPoint()`, que retorna as coordenadas do rato.

`translatePoint(int x, int y)` permite alterar a localização do evento.

Outra função interessante é `getButton()`, que nos permite saber qual botão causou o evento. Retorna uma constante `int NOBUTTON`, `BUTTON1`, `BUTTON2`, `BUTTON3`. `NOBUTTON` indica que nenhum botão foi premido ou libertado.

MOUSE_CLICKED	The user clicked the mouse.
MOUSE_DRAGGED	The user dragged the mouse.
MOUSE_ENTERED	The mouse entered a component.
MOUSE_EXITED	The mouse exited from a component.
MOUSE_MOVED	The mouse moved.
MOUSE_PRESSED	The mouse was pressed.
MOUSE_RELEASED	The mouse was released.
MOUSE_WHEEL	The mouse wheel was moved.

`MouseEvent(Component src, int type, long when, int modifiers, int x, int y, int clicks, boolean triggersPopup)`



MouseWheelEvent

Subclasse de MouseEvent, esta encapsula eventos específicos da roda do rato. Pode ser WHEEL_BLOCK_SCROLL ou WHEEL_UNIT_SCROLL, sendo distinguido na variável scrollHow.

```
MouseWheelEvent(Component src, int type, long when, int modifiers,  
    int x, int y, int clicks, boolean triggersPopup,  
    int scrollHow, int amount, int count)
```

Podemos saber quantas unidades de rotação a roda teve com getWheelRotation() ou getPreciseWheelRotation() para ratos com rodas de alta resolução, retornando um double. Com getScrollType() obtemos o nosso scrollHow(), caso seja WHEEL_UNIT_SCROLL então podemos obter o número de unidades a dar scroll através de getScrollAmount().



WindowEvent

Existem 10 diferentes tipos de WindowEvents. Esta classe é subclasse de ComponentEvent e tem diversos construtores.

O método getWindow() permite-nos obter a janela que gerou o evento, getOppositeWindow() dá-nos a outra janela, quando existe uma alteração de foco ou ativação. getOldState() e getNewState() dão os estados anteriores e posteriores ao evento.

WindowEvent(Window src, int type)

WindowEvent(Window src, int type, Window other)

WindowEvent(Window src, int type, int fromState, int toState)

WindowEvent(Window src, int type, Window other, int fromState, int toState)

WINDOW_ACTIVATED	The window was activated.
WINDOW_CLOSED	The window has been closed.
WINDOW_CLOSING	The user requested that the window be closed.
WINDOW_DEACTIVATED	The window was deactivated.
WINDOW_DEICONIFIED	The window was deiconified.
WINDOW_GAINED_FOCUS	The window gained input focus.
WINDOW_ICONIFIED	The window was iconified.
WINDOW_LOST_FOCUS	The window lost input focus.
WINDOW_OPENED	The window was opened.
WINDOW_STATE_CHANGED	The state of the window changed.



Interfaces de Listeners

Interface	Description
ActionListener	Defines one method to receive action events.
AdjustmentListener	Defines one method to receive adjustment events.
ComponentListener	Defines four methods to recognize when a component is hidden, moved, resized, or shown.
ContainerListener	Defines two methods to recognize when a component is added to or removed from a container.
FocusListener	Defines two methods to recognize when a component gains or loses keyboard focus.
ItemListener	Defines one method to recognize when the state of an item changes.
KeyListener	Defines three methods to recognize when a key is pressed, released, or typed.
MouseListener	Defines five methods to recognize when the mouse is clicked, enters a component, exits a component, is pressed, or is released.
MouseMotionListener	Defines two methods to recognize when the mouse is dragged or moved.
MouseWheelListener	Defines one method to recognize when the mouse wheel is moved.
TextListener	Defines one method to recognize when a text value changes.
WindowFocusListener	Defines two methods to recognize when a window gains or loses input focus.
WindowListener	Defines seven methods to recognize when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.



Adapter Classes

Existe um conjunto de classes que permite simplificar a criação de handlers. Estas fornecem uma implementação vazia de todos os métodos de uma interface de event listener. São úteis quando queremos receber e processar apenas alguns eventos implementados por uma interface em particular. Podemos definir uma nova classe para agir como event listener ao fazer extend a uma destas classes de adaptação e implementando apenas os eventos a tratar.

Adapter Class	Listener Interface
ComponentAdapter	ComponentListener
ContainerAdapter	ContainerListener
FocusAdapter	FocusListener
KeyAdapter	KeyListener
MouseAdapter	MouseListener, MouseMotionListener, and MouseWheelListener
MouseMotionAdapter	MouseMotionListener
WindowAdapter	WindowListener, WindowFocusListener, and WindowStateListener